

Magenta'nın promptlarında ne eksik? Claude Code ile karşılaştırma

Magenta 0.11.1'in 80 promptunu, sızdırılmış Claude Code system promptlarıyla (Opus 4.6, 4.8, Opus 5, Fable 5) karşılaştırdım. Amacım iki sorunu açıklamak: turların 2–3 kat uzun sürmesi ve kodun iki kat uzun olmasına rağmen özellikle UI işlerinde hata vermesi.

HAZIRLAYAN	Claude Code — M Ali Ozturk için
TARİH	28 Temmuz 2026
İNCELENEN SÜRÜM	Magenta 0.11.1.0 (80 prompt, 29 tool)
REFERANS	claude-code-opus-4.6, -4.7, -4.8, -opus-5, -fable-5
KAYNAK	system_prompts_leaks-main.zip
KAPSAM	Sadece okuma ve analiz. Hiçbir prompt değiştirilmedi.

1 Kısa özet

Ana fark promptların kalitesi değil. Ana fark, bilginin nerede durduğu.

Magenta, "nasıl çalışılır" bilgisini system prompt'un içine yazıyor. Bu metin her istekte modele gidiyor. Ayrıca her turun sonunda ek kontroller çalıştırıyor.

Claude Code aynı bilgiyi **tool description**'ların ve **skill**'lerin içine koyuyor. Bu metinler sadece o tool veya skill gerektiğinde okunuyor.

Sonuç: Magenta bu bilginin bedelini her istekte ve her turda ödüyor. Claude Code sadece gerektiğinde ödüyor. İki sorunun da büyük kısmı bu tek tercihten geliyor.

HER İSTEKTE GÖNDERİLEN SABİT MALİYET

Ajan	Talimatlar	Tool açıklamaları	Toplam	Oran	Tool
Claude Code · Opus 5	2.975	20.503	23.478	%13 / %87	33
Claude Code · Fable 5	3.295	20.500	23.795	%14 / %86	33
Claude Code · Opus 4.6	8.185	24.190	32.375	%25 / %75	32
Magenta 0.11.1	4.500	4.462	8.962	%50 / %50	29

Sayılar token cinsindendir. Tahmin için 3,5 karakter = 1 token kuralı kullanıldı. Bu, Magenta'nın kendi context sayacının kullandığı kuralın aynısı. Claude Code rakamlarına JSON şemaları dahil değil; iki tarafta da sadece düz metin sayıldı.

Tabloyu nasıl okumalı: Claude Code, bütçesinin yaklaşık %87'sini tool açıklamalarına harcıyor. Magenta ise yarı yarıya bölüyor. Yani Claude Code modele "genel olarak nasıl davranmalısın" demek yerine, "bu tool'u tam olarak ne zaman ve nasıl kullanmalısın" diyor.

BEŞ BULGU, ETKİSİNE GÖRE SIRALI

- Her turda ödenen ekstra bir vergi var. Claude Code'da bu hiç yok.** Magenta her iş turunda modele fazladan bir soru soruyor. Bu, fazladan bir model çağırısı demek. Tek bir görev etrafında bu sayı 4'e kadar çıkabiliyor. **YAVAŞLIK**
- Çalışma talimatları yanlış yerde duruyor.** Magenta'nın `system.working-method` bölümü tek başına 1.404 token ve her istekte gidiyor. Claude Code aynı bilgiyi `TaskCreate` ve `TaskUpdate` açıklamalarına koymuş. O açıklamalar Magenta'nınkinin 5,5 ve 5,9 katı — ama sadece plan yapılırken okunuyor. **MİMARİ**
- Kapanış özeti çok uzun isteniyor.** Magenta her görevin sonunda açık bir "başarı karşılaştırması" ve hüküm istiyor. Claude Code ise "bir iki cümle, başka bir şey yok" diyor. **UZUNLUK**
- UI değişikliğini gözle doğrulamanın yolu yok.** Claude Code 4.6 "dev server'ı başlat ve tarayıcıda dene" diyordu. 5.x sürümünde bu bir `run` skill'ine taşındı. Magenta'da ikisi de yok. Üstelik bir prompt modele "burada gözün yok" diyor. **UI HATALARI**

5. **Hiçbir şey ertelenmiyor.** Claude Code 30 hazır skill ile geliyor. Hiçbiri çağrılana kadar context'te yer kaplamıyor.

Magentra'da özel bilgi ya hep açık ya da hiç yok.

MİMARİ

2 Yöntem ve uyarılar

Beş farklı Claude Code sürümünün system promptunu okudum. Ayrıca yanlarında gelen skill'leri, subagent tanımlarını ve enjekte edilen hatırlatmaları inceledim. Magentra tarafındaki bütün sayıları ise çalışan motordan canlı olarak aldım. Yani bunlar kaynak kodda yazan değil, gerçekten gönderilen değerler.

Üç uyarı önemli. Birincisi, sızıntı dosyaları resmi yayın değil, topluluk kayıtları. Bir bölüm eski veya eksik olabilir. İkincisi, token sayıları gerçek bir tokenizer ile değil, tahminle hesaplandı. Oranlara güvenin, mutlak sayıları yaklaşık kabul edin. Üçüncüsü, Claude Code kendi üreticisinin modelleriyle çalışıyor. Bazı davranışlar prompttan değil, modelin eğitiminden geliyor olabilir.

3 Bulgu 1 — Her turda ekstra model çağrısı

YAVAŞLIK

Turların 2–3 kat uzun sürmesinin en doğrudan açıklaması bu.

Magentra her iş turunun sonunda bir "self-verify" adımı çalıştırıyor. Modele soruyor: "İş bitti mi? Bittiyse sadece **DONE** yaz." Bu cevap ücretsiz değil. Tam bir model çağrısıdır — system prompt, bütün tool'lar ve bütün konuşma tekrar gönderilir. Ve turda hiçbir şey değişmese bile çalışır.

Bunun üstüne başka kontroller de var. Dosya değiştirip hiç komut çalıştırmadıysanız 654 token'lık bir hatırlatma gelir; bu bir çağrı daha demektir. Sadece kendi yazdığınız mock'larla test ettiyseniz 450 token'lık başka bir hatırlatma gelir. Açık uçlu bir istekte ise iş başlamadan önce "clarify" katmanı devreye girer.

TEK BİR GÖREV ETRAFINDAKİ EKSTRA MODEL ÇAĞRILARI

Ne zaman	Magentra	Prompt boyu	Claude Code
Açık uçlu istek gelince	clarify katmanı → +1 çağrı	786	yok
Her iş turunda, koşulsuz	self-verify → +1 çağrı	255 + 126	yok
Kod değişti, hiç komut çalışmadı	runtime-evidence → +1 çağrı	654	yok
Sadece kendi mock'uyla test edildi	circular-evidence → +1 çağrı	450	yok
Çalıştı ama özet yazmadı	wrap-up uyarısı → +1 çağrı	70	yok
En kötü durum	4 ekstra tam çağrı		0

Claude Code'un da enjekte edilen hatırlatmaları var — tam yedi tane. Ama hepsi bir *olaya* bağlı: bir ekibe katılma, **/model** ile model değiştirme, container'ın yeniden başlaması, plan modunun açılması. Hiçbiri her turda çalışmıyor.

Claude Code "işim bitti mi?" sorusunu tek bir cümleyle çözüyor. Bu cümle zaten system prompt'un içinde olduğu için ekstra hiçbir maliyeti yok:

Before ending your turn, check your last paragraph. If it is a plan, an analysis, a question, a list of next steps, or a promise about work you have not done ('I'll...'), do that work now with tool calls.

Turu bitirmeden önce son paragrafına bak. Eğer orada bir plan, bir soru veya "şunu da yaparım" gibi bir söz varsa, o işi şimdi yap.

claude-code-fable-5.md — Context management

Magentra da `system.autonomy` içinde neredeyse aynı şeyi söylüyor. Fark şu: Magentra bunu söyledikten *sonra* bir de modele sorup teyit ettiriyor. Maliyet o teyit çağrısında, metinde değil.

NE YAPMALI

En yüksek getirili tek değişiklik bu. Self-verify adımını koşullu yapın: sadece turda dosya değiştiyse veya açık task kaldıysa çalışsın. Böylece sohbet turlarından ve sadece okuma yapılan turlardan bir tam çağrı silinir, ama gerçekten gerekli olduğu yerde kontrol korunur.

4 Bulgu 2 — Talimatlar yanlış yerde

MİMARİ

İki tarafta da bulunan 24 tool'u karşılaştırdım. Sonuç ikiye ayrılıyor ve çok net.

TOOL AÇIKLAMA UZUNLUKLARI (KARAKTER)

Tool	Magentra	Claude Code	Kat	Tür
Monitor	601	4.970	8,3×	koordinasyon
Workflow	2.112	15.602	7,4×	koordinasyon
EnterWorktree	460	3.242	7,0×	koordinasyon
TaskUpdate	343	2.019	5,9×	planlama
Skill	248	1.423	5,7×	koordinasyon
TaskCreate	390	2.163	5,5×	planlama
AskUserQuestion	429	1.804	4,2×	etkileşim
Bash	726	1.374	1,9×	çalıştırma
Agent	1.110	1.584	1,4×	koordinasyon
Read	835	796	1,0×	dosya
Edit	433	366	0,8×	dosya
Write	315	247	0,8×	dosya

Dosya tool'larınız iyi durumda. Sorun koordinasyon tool'larında. Read, Edit ve Write, Claude Code'unkiyle aynı boyutta veya daha uzun. Ama işin *nasıl organize edileceğini* belirleyen her tool 4 ila 9 kat daha zayıf. Claude Code tam olarak o açıklamaların içine şunları yazıyor: ne zaman plan yapılır, ne zaman yapılmaz, ne zaman komut arka plana atılır, ne zaman subagent açılır.

Magentra'da bu bilgi var. Ama yanlış yerde: her istekte gönderilen 1.404 token'lık `system.working-method` bölümünde. Yani sadece "bu fonksiyon ne yapıyor?" diye sorulduğunda bile gidiyor. İki tarafın "ne zaman task listesi aç" tarifini yan yana koyalım:

MAGENTRA · SYSTEM.WORKING-METHOD (HEP AÇIK)

"Birkaç adımdan fazla süren her iş için: hiçbir şeyi düzenlemeden önce TaskCreate ile parçala — her somut adım için bir task — ve son task bir doğrulama task'ı olsun, çalıştıracağın komutu ve başarı çıktısını yazarak."

Her istekte gidiyor. Tek satırlık sorularda bile.

CLAUDE CODE · TASKCREATE AÇIKLAMASI

"3 veya daha fazla ayrı adım gerektiren karmaşık işlerde kullan... **Bu tool'u NE ZAMAN KULLANMA:** tek ve basit bir iş varsa; iş önemsizse; 3 önemsiz adımdan kısa sürüyorsa; iş sadece sohbetse."

Sadece TaskCreate düşünülürken okunuyor.

İki fark var. Claude Code net bir sayı veriyor: **3 adım**. Ve "ne zaman kullanma" kısmına en az "ne zaman kullan" kadar yer ayırıyor. Magentra "birkaç adımdan fazla" diyor ve olumsuz durumu başka bir bölümde tek satırda geçiriyor.

"Kod neden iki kat uzun?" sorusunun en olası cevabı burada. Her istekte "parçala, her task'ı bağımsız bir issue gibi yaz, doğrulama task'ı ekle, sistemin tasarımını değerlendir, tasarım borcunu kaydet" denen bir ajan, 15 satırlık bir düzeltmede de bunların hepsini yapar.

5 Bulgu 3 — Kapanış özeti

UZUNLUK

MAGENTRA · SYSTEM.WORKING-METHOD

"kullanıcı için kısa bir kapanış yaz: ne yapıldı veya değişti (dosyalar), nasıl kullanılır, ve bir başarı karşılaştırması — doğrulama task'ı neyi bekliyordu, sen ne gözlemledin, ve açık bir hüküm (kriterler karşılandı mı) — artı ters giden veya açık kalan ne varsa. Bir iş turunu asla sessizce bitirme."

Beş zorunlu parça. Adı "kısa", tarifi bir rapor.

CLAUDE CODE · OPUS 4.6

"End-of-turn summary: one or two sentences. What changed and what's next. Nothing else."

Türkçesi: Tur sonu özeti: bir iki cümle. Ne değişti ve sırada ne var. Başka bir şey yok.

Magentra'nın `system.communication` bölümü de "cevabın boyutunu sorunun boyutuna göre ayarla" diyor. Ama `system.working-method` boyuttan bağımsız olarak beş parça istiyor. Ayrıca "asla sessizce bitirme" diye ekliyor. İki bölüm çeliştiğinde genelde daha spesifik olan kazanır — burada da raporu isteyen taraf kazanıyor.

6 Bulgu 4 — UI için "gözün yok" deniyor

UI HATALARI

UI işlerinin neden hata verdiğine en doğrudan bakan bulgu bu.

Claude Code 4.6'da system prompt'un içinde açık bir kural vardı:

For UI or frontend changes, start the dev server and use the feature in a browser before reporting the task as complete... Type checking and test suites verify code correctness, not feature correctness.

UI veya frontend değişikliklerinde, işi bitti demeden önce dev server'ı başlat ve özelliği tarayıcıda kullan. Type check ve test suite kodun doğruluğunu ölçer, özelliğin doğruluğunu değil.

claude-code-opus-4.6.md

Bu cümle 4.8, Opus 5 ve Fable 5'te system prompt'tan çıkarıldı. Ama silinmedi — bir skill'e taşındı. `run` adlı bu skill şöyle başlıyor: "Çalıştırmak demek, gerçek uygulamayı başlatmak ve onunla etkileşmek demektir — test suite değil, bir iç fonksiyonu import edip `console.log` yazmak da değil." Skill, proje tipine göre yol gösteriyor ve Electron listedeki tiplerden biri.

Magentra'da ne bu kural var ne de bu skill. Onun yerine şu var:

Judge against something you can actually read: exit codes, stdout, a log line... **You have no eyes here — never claim you looked at a screenshot or a window.**

*Gerçekten okuyabileceğin bir şeye bak: exit code, stdout, log satırı... **Burada gözün yok — asla bir ekran görüntüsüne veya pencereye baktığını söyleme.***

Magentra — finishing.runtime-evidence

Niyet doğru: ajan görmediği şeyi gördüm demesin. Ama cümle yanlış şeyi söylüyor. Dürüstlük kuralı olması gerekirken bir *yetenek* beyanı olmuş. Oysa workbench ekran görüntüsü alabiliyor.

Sonuç şu: "gözüm yok" diyen bir ajan uygulamayı açmayı denemez. Type check'e döner. Ve hiç görmediği bir UI değişikliği için "başarılı" der. Gözlenen hata tam olarak budur.

Aynı prompt sorunu bir kat daha büyütüyor. İçindeki kaçış maddesi, "ekran, cihaz veya konsol gerektiren" bir değişiklikte durmayı meşru sayıyor. Masaüstü bir UI uygulamasında bu madde neredeyse her frontend işinde devreye girer.

NE YAPMALI

İki ucuz değişiklik. Birincisi, "gözün yok" cümlesini gerçek anlamına daraltın: yapmadığın bir gözlemi asla iddia etme. Yeteneğin yok demeyin. İkincisi, bu repo için bir `run` skill'i ekleyin — Electron workbench'i headless başlatıp bir kare yakalasin. Böylece runtime-evidence adınının UI işlerinde gösterecek gerçek bir yolu olur.

7 Bulgu 5 — Hiçbir şey ertelenmiyor

MİMARİ

Claude Code 30 hazır skill ile geliyor. Hiçbiri çağrılana kadar context'te yer kaplamıyor; model sadece tek satırlık bir liste görüyor. Bunlardan bazıları Magentra'da hiç yok:

MAGENTRA'DA KARŞILIĞI OLMAYAN CLAUDE CODE SKILL'LERİ

Skill	Ne yapıyor	Magentra'da
<code>run</code>	Gerçek uygulamayı başlatıp değişikliği görmek	yok
<code>simplify</code>	Sonradan sadeleştirme ve tekrar kullanım taraması	yok
<code>code-review</code>	Çalışma diff'inin yapılandırılmış incelemesi	yok
<code>security-review</code>	Bekleyen değişikliklerde güvenlik taraması	yok
<code>debug</code>	Sistematik hata ayıklama yordamı	discipline skill var

Bir örüntü var ve adını koymak gerekiyor. Claude Code'un hep açık prompt'u sürümler boyunca *küçüldü*: Opus 4.6'da 8.185 token, Opus 5'te 2.975. Aynı sürede tool açıklamaları ve skill kütüphanesi büyüdü. Yani bilgi "her zaman

burada"dan "gerektiğinde burada"ya taşındı. Magentra şu anda Claude Code'un iki nesil önceki oranında — hatta biraz daha gerisinde.

8 Magentra'nın önde olduğu yerler

Adil olmak gerekirse: Magentra her yerde geride değil. Bazı özelliklerinin Claude Code'da hiç karşılığı yok.

- **GraphQL.** Claude Code'da yapısal sorgu tool'u yok. Kodu `find`, `grep` ve bir arama subagent'i ile buluyor. O subagent kendi tanımına göre "dosyaların tamamını değil, parçalarını okuyor". Magentra'nın import grafiği sorguları — slice, blast radius, structure — Claude Code'un ancak tahmin edebildiği etki sorularını cevaplıyor. Bu gerçek bir üstünlük ve promptlarda *az kullanılıyor*: working-method içinde tek satır geçiyor.
- **Tasarım atlası.** Ajanın kendi yazıp güncellediği bir `.magentra/ATLAS.md`, sadece okunan bir `CLAUDE.md` 'den daha güçlü bir proje hafızasıdır.
- **Crew ve backpack.** Kendi indekslenmiş bilgi dokümanları olan uzman ajanlar (BM25 + embedding). Claude Code'un subagent'leri hafızasız ve bilgisiz.
- **Mission ve cron.** Workspace dosyası olarak tanımlanan zamanlanmış otonom iş.
- **Prompt registry.** Bu haftaki çalışmadan sonra Magentra'nın gönderdiği her prompt tek bir yerden listelenebiliyor ve canlı düzenlenebiliyor. Claude Code'un promptları bir binary'nin içinde derli. Bu raporun yazılablmesinin tek sebebi, birinin onları proxy ile yakalamış olması.

9 Öneriler

Beklenen etkiye göre sıralı. Her satır Prompt Lab'de açılacak prompt id'sini veriyor. Böylece her biri ayrı ayrı denenip ölçülebilir.

ÖNCELİKLİ DEĞİŞİKLİKLER

#	Değişiklik	Prompt / alan	Hedef
1	Self-verify adımını koşullu yap — turda dosya değişmediyse ve task kapanmadıysa çalışmasın	finishing.self-verify	yavaşlık
2	Kapanış özetini kısalt — beş parçalı başarı karşılaştırması yerine iki cümle; hüküm sadece doğrulama task'ı varsa	system.working-method	uzunluk
3	"Gözün yok" cümlesini dürüstlük kuralına daralt; Electron yolunu gözlem yöntemi olarak ekle	finishing.runtime-evidence	UI hataları
4	Uygulamayı headless başlatıp kare yakalayan bir <code>run</code> skill'i ekle	yeni skill	UI hataları
5	Planlama talimatını working-method'dan çıkarıp TaskCreate/TaskUpdate içine taşı; net 3 adım eşiği ve açık bir "ne zaman kullanma" ekle	tool.TaskCreate tool.TaskUpdate	uzunluk, yavaşlık
6	Clarify katmanını daha ucuz bir teste bağla veya "rutin kararları kendin ver" diyen tek bir cümleyle değiştir	clarify.system	yavaşlık
7	Bir "düzeltme" bölümü ekle — özür girişi yok, gereksiz kendini düzeltme yok, sadece kullanıcının kararını değiştiren hatayı düzelt	system.communication	uzunluk
8	Kapsam sadakati ekle — istenen kapsam teslimattır; sessizce daraltma, genişletme veya dönüştürme	system.autonomy	doğruluk
9	Bash, Monitor ve Agent açıklamalarını kalınlaştır — şu an prose'da ima edilen arka plan ve devretme kurallarını oraya yaz	tool.Bash, tool.Monitor, tool.Agent	yavaşlık
10	Büyük veya geri alınamaz işler için bir plan modu geri getir; ya da careful mode'un neden yerine bir şey konmadan kaldırıldığını yaz	motor	doğruluk

NASIL ÖLÇERİZ

Yukarıdaki her öneri bir hipotezdir. Dürüst olmak gerekirse hiçbir Magentra için henüz kanıtlanmadı. Prompt Lab'in `--dir` parametresi bir oturumu ayrı bir override setine yönlendiriyor. O yüzden yöntem şu olmalı:

- Küçük ve sabit bir görev seti belirleyin. İçinde en az iki UI görevi olsun — en zayıf alan orası.
- Aynı görevleri önce mevcut promptlarla çalıştırın.
- Sonra **tek seferde tek bir promptu** değiştirip tekrar çalıştırın.
- Her koşuda şunları kaydedin: tur sayısı, geçen süre, çıktı token'ı, ve sonucun gerçekten çalışıp çalışmadığı.

Hepsini aynı anda değiştirirseniz daha iyi bir ajan elde edebilirsiniz ama kimse nedenini açıklayamaz. Prompt setleri bakımı imkânsız hale böyle gelir.

Önce test edilmeye değer tek bir tahmin: **1 numaralı öneri tek başına her turdan bir tam model çağrısı siler.** Eğer 2–3 katlık fark çoğunlukla çağrı sayısından geliyorsa — çağrı başına token'dan değil — bu tek değişiklik farkın görünür bir kısmını tek başına kapatmalı.